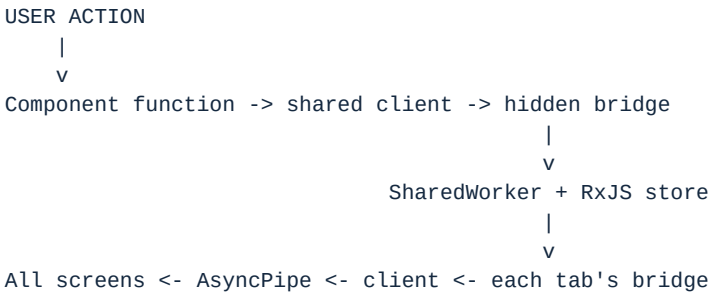
# Angular A, B and C Complete data flow

## A practical demo guide with code, values and the next function to follow.

Read one scenario at a time. Each numbered step names the actual source file, explains what runs, shows the code or message, and tells you where execution continues.

Project A adds and completes tasks and shows both counters. Project B edits tasks and changes counter B. Project C deletes tasks and changes counter C.




Native Federation shares the client implementation. Browser messages connect the tabs. One worker owns the common todo/counter data. Each page receives its own copy for display.

Prepared from the current source on 14 October 2026. Code panels show relevant source excerpts; comments and unrelated branches may be omitted. Message values such as req-A-1 are illustrative: the application generates UUIDs.

The steps describe execution order, not intentional delays. Normal function calls run directly. postMessage crosses an asynchronous browser boundary.

# Reading map and file roles

## Startup

Pages 3-4: how Angular starts, federation loads the client, and the worker connection is created.

## Add a todo

Pages 5-10: all 15 steps from typing to updating every screen and confirming the action.

## Edit and delete

Pages 11-14: local editing, save, deletion, messages, worker changes and the return path.

## Counters

Pages 15-19: complete increment flow, decrement/reset examples and simultaneous updates.

## Completion, errors and demo

Pages 20-22: checkbox changes, failures, data lifetime and a presentation checklist.

## The files you will follow

```
Project-A/src/app/todo.ts           Add / complete actions
Project-B/src/app/app.ts            Edit / counter B actions
Project-C/src/app/app.ts            Delete / counter C actions
Project-A/src/app/app-state.service.ts
    Shared client: send actions, receive data, expose state$
Project-A/public/state-bridge.js
    Forward messages between each page and the worker
Project-A/public/state-worker.js
    Validate and modify the common data, then broadcast it
```

A filename beginning with Project-A can run inside B or C: federation loads the client code into the consuming page, and each page embeds an A-origin bridge. The source owner and the execution location are different concepts.

# Startup | Angular and federation

## Start 1 - Prepare the page

File: Project-A/src/main.ts

All three projects follow this main.ts pattern. Federation initializes before the dynamic bootstrap import. The first catch logs an error and still allows the bootstrap attempt.

```
initFederation()
  .catch(err => console.error(err))
  .then(() => import('./bootstrap'))
  .catch(err => console.error(err));
```

Next: src/bootstrap.ts calls bootstrapApplication(App, appConfig), which creates the root Angular component.

## Start 2 - B/C load A's client

File: Project-B/src/app/app.ts

Angular calls ngOnInit(). B and C request A's metadata URL and the ./AppState exposure. A imports its client directly instead.

```
const module = await loadRemoteModule({
  remoteEntry: 'http://localhost:4200/remoteEntry.json',
  exposedModule: './AppState',
});
this.appState.set(module.appState);
```

Next: Project-A/federation.config.js maps ./AppState to src/app/app-state.service.ts. Evaluating that module creates appState.

```
export const appState = new AppStateClient();
```

The metadata identifies the generated module and shared dependencies. Federation loads code, not a todo API response. singleton dependency sharing does not share a JavaScript object between tabs.

# Startup | Connect to the common store

## Start 3 - Create the hidden bridge

File: Project-A/src/app/app-state.service.ts

AppStateClient.constructor() creates a hidden iframe. Its URL points to A even when the client runs in B or C.

```
this.bridge = document.createElement('iframe');
this.bridge.hidden = true;
this.bridge.src = `${projectAOrigin}/state-bridge.html`;
this.bridge.onload = () => {
  this.bridge.contentWindow?.postMessage(
    { channel, type: 'connect' }, projectAOrigin);
};
```

Next: Project-A/public/state-bridge.html loads state-bridge.js; its message listener calls connect().

## Start 4 - Connect and receive initial data

File: Project-A/public/state-bridge.js

Each bridge is on A's origin and uses the same worker URL/name. In the supported same-profile localhost setup, these connections reach the common worker.

```
worker = new SharedWorker('/state-worker.js', 'project-a-rxjs-store');
worker.port.onmessage = event => sendToParent(event.data);
```

Next: state-worker.js -> self.onconnect registers the port and sends store.value to the new tab.

```
port.postMessage({ type: 'state', state: store.value });
```

A new worker starts with { countB: 0, countC: 0, todos: [] }. A later tab receives the current data, not a fresh empty store. The reply follows the return path explained in Add steps 11-14.

# Add a todo | Type and submit

## Add 1 - Type the text

File: Project-A/src/app/todo.html

Typing "Prepare demo" changes newTask in A's component. It does not yet change the worker store.

```
<input id="new-task" name="newTask"
       [(ngModel)]="newTask" placeholder="Enter a task" />

<!-- Component value after typing: -->
newTask = 'Prepare demo';
```

Next: Click Add. The form's (ngSubmit)="addTask()" calls the component method.

## Add 2 - Pass text to the client

File: Project-A/src/app/todo.ts

addTask() passes the current input value to addTodo(). It clears the input only if the client accepts the send.

```
addTask() {
  if (appState.addTodo(this.newTask)) this.newTask = '';
}

// Value passed:
appState.addTodo('Prepare demo');
```

Next: Project-A/src/app/app-state.service.ts -> addTodo(text).

## Add 3 - Create the action

File: Project-A/src/app/app-state.service.ts

The client describes what the user wants. It does not insert a local task.

```
addTodo(text: string) {
  return this.send({ type: 'addTodo', text });
}
```

Next: send() receives { type: "addTodo", text: "Prepare demo" }.

# Add a todo | Send through the bridge

## Add 4 - Track and send the request

File: Project-A/src/app/app-state.service.ts

send() rejects a disconnected/busy client. Otherwise it creates an ID, publishes pending:true, starts a reply timer and posts to the iframe. The return true means sent for processing, not already saved.

```
  private send(action: StateAction) {
    if (!this.view.connected || this.view.pending) return false;
    this.pendingRequest = crypto.randomUUID();
    this.publish({ ...this.view, pending: true, error: '' });
    this.waitForReply();
    this.bridge.contentWindow?.postMessage({
      channel, type: 'action', requestId: this.pendingRequest, action,
    }, projectAOrigin);
    return true;
  }
```

Next: The browser delivers the action asynchronously to state-bridge.js.

## The message being passed

```
{
  channel: 'project-a-state', type: 'action', requestId: 'req-A-1',
  action: { type: 'addTodo', text: 'Prepare demo' }
}
```

## Add 5 - Forward to the worker

File: Project-A/public/state-bridge.js

The window message listener checks the parent window, allowed origin and channel. For an action it forwards the message unchanged.

```
if (message.type === 'action' && worker) {
  worker.port.postMessage(message);
}
```

Next: state-worker.js -> port.onmessage receives event.data asynchronously. The bridge never modifies the task.

# Add a todo | Worker receives and validates

## Add 6 - Extract the action

File: Project-A/public/state-worker.js

The worker checks the action envelope and calls applyAction with the inner action. Only after it succeeds does it send done to the requester.

```
const message = event.data;
// After the envelope checks:
applyAction(message.action);

// applyAction receives:
{ type: 'addTodo', text: 'Prepare demo' }
```

Next: applyAction(action) in the same worker file.

## Add 7 - Copy current state

File: Project-A/public/state-worker.js

The function copies store.value before making changes. This keeps a failed validation from partially changing the saved data.

```
const state = structuredClone(store.value);

// Starting data in this example:
{ countB: 0, countC: 0, todos: [] }
```

Next: The switch selects case addTodo and calls taskText(action.text).

## Add 8 - Validate the text

File: Project-A/public/state-worker.js

taskText trims spaces and rejects blank or non-string input. The cleaned string returns to applyAction.

```
function taskText(text) {
  if (typeof text !== 'string' || !text.trim())
    throw new Error('Task text cannot be blank.');
  return text.trim();
}
```

Next: The addTodo branch receives "Prepare demo" and creates the task.

# Add a todo | Create and publish

## Add 9 - Insert the task into the copy

File: Project-A/public/state-worker.js

The worker assigns an ID from nextTodoId and sets completed:false. IDs are allocated centrally, not separately in each tab.

```
state.todos.push({ id: nextTodoId++, text, completed: false });

// Copied state is now:
{
  countB: 0, countC: 0,
  todos: [{ id: 1, text: 'Prepare demo', completed: false }]
}
```

Next: applyAction() finishes the switch and calls store.next(state).

## Add 10 - Commit and broadcast

File: Project-A/public/state-worker.js

BehaviorSubject remembers the new state. Its subscription runs synchronously and queues a state message to every connected port.

```
store.next(state);

// Broadcast subscription, with error handling omitted here:
store.subscribe(state => {
  for (const client of clients) {
    client.postMessage({ type: 'state', state });
  }
});
```

Next: Each bridge receives { type: "state", state: updatedState }. Cross-context delivery is asynchronous.

The subscriber is registered once when the worker starts. It is not registered again for every action. A new tab receives store.value explicitly in self.onconnect.

# Add a todo | Receive the updated data

## Add 11 - Bridge replies to its page

File: Project-A/public/state-bridge.js

The bridge receives the worker message on its port. sendToParent adds the channel and sends it to the validated parent origin.

```
worker.port.onmessage = event => sendToParent(event.data);

function sendToParent(message) {
  if (parentOrigin) {
    window.parent.postMessage({ channel, ...message }, parentOrigin);
  }
}
```

Next: Each tab's client receives a window message in receive(event).

## Add 12 - Validate the snapshot

File: Project-A/src/app/app-state.service.ts

receive checks the origin, source iframe and channel first. For a valid state message it checks counters/tasks and adds local UI status. This does not finish an outstanding request.

```
if (message.type === 'state' && this.validState(message.state)) {
  this.publish({
    ...message.state, connected: true,
    pending: this.view.pending, error: ''
  });
  if (!this.pendingRequest) clearTimeout(this.replyTimeout);
}
```

Next: publish(view) receives the tasks/counters plus connected, pending and error.

validState checks safe-integer counters and task IDs, an array of tasks, string text and boolean completion. Invalid snapshots are ignored. The worker owns business data; connected/pending/error belong to each client.

# Add a todo | Render and confirm

## Add 13 - Emit through state$

File: Project-A/src/app/app-state.service.ts

publish remembers the received view and invokes emit. The observable's emit callback calls subscriber.next(view). shareReplay gives local template subscribers the latest value.

```
private publish(view: AppState) {
  this.view = view;
  this.emit?.(view);
}
// Set when the observable source is subscribed:
this.emit = view => subscriber.next(view);
```

Next: AsyncPipe receives the emission in the template.

## Add 14 - Display the todo

File: Project-A/src/app/todo.html

AsyncPipe supplies the latest state and marks the view for checking. Angular renders the list. B and C use service.state$ | async and receive the same business data.

```
@if (state$ | async; as state) {
  @for (task of state.todos; track task.id) {
    <span [class.completed]="task.completed">{{ task.text }}</span>
  }
}
```

Next: All connected screens show "Prepare demo"; the requester also receives a confirmation.

## Add 15 - Finish the pending request

File: Project-A/public/state-worker.js

After applyAction succeeds, the worker queues done only to the requesting port. It travels through bridge sendToParent -> client receive. A matching request ID clears pending and the timer.

```
port.postMessage({ type: 'done', requestId: message.requestId });
// Example reply: { type: 'done', requestId: 'req-A-1' }
```

Next: A's controls become available again. Browser paint timing can differ between tabs.

# Edit in B | Local draft, then Save

## Edit 1 - Open the editor

File: Project-B/src/app/app.ts

The Edit button in app.html calls startEditing(task). Copying the text lets you type without changing the common task.

```
startEditing(task: TodoItem) {
  this.editingId = task.id;
  this.draftText = task.text;
}
// Local values: editingId = 1; draftText = 'Prepare demo';
```

Next: The edit input binds draftText through ngModel. Type "Present demo".

## Edit 2 - Submit the draft

File: Project-B/src/app/app.ts

Save calls saveTask(task.id). Blank text is not sent. An accepted send closes the local editor; confirmation comes later.

```
saveTask(id: number) {
  const text = this.draftText.trim();
  if (text && this.appState()?.editTodo(id, text)) this.cancelEditing();
}

cancelEditing() {
  this.editingId = null;
  this.draftText = '';
}
```

Next: The loaded client's editTodo(1, "Present demo") packages the edit action.

Clicking Cancel directly calls cancelEditing without sending any edit. A and C keep the saved text while you type. Switching to an already connected B tab does not reload the common data.

# Edit in B | Worker change and return

## Edit 3 - Send the ID and new text

File: Project-A/src/app/app-state.service.ts

editTodo calls send. The request crosses the same bridge and worker boundaries as Add: send -> bridge message listener -> worker port.onmessage -> applyAction.

```
editTodo(id: number, text: string) {
  return this.send({ type: 'editTodo', id, text });
}
// Inner action:
{ type: 'editTodo', id: 1, text: 'Present demo' }
```

Next: The worker selects case editTodo.

## Edit 4 - Replace only the task text

File: Project-A/public/state-worker.js

findTask searches the copied state by ID and throws if missing. taskText validates the new text. The original ID and completion flag stay unchanged.

```
case 'editTodo':
  findTask(state, action.id).text = taskText(action.text);
  break;

// Before: { id: 1, text: 'Prepare demo', completed: false }
// After:  { id: 1, text: 'Present demo', completed: false }
store.next(state);
```

Next: store.subscribe sends the updated snapshot to every bridge.

## Edit 5 - Complete return path

```
Worker store.subscribe -> bridge port.onmessage -> sendToParent
  -> client receive -> validState -> publish -> state$ -> AsyncPipe
  -> A, B and C display "Present demo".
Worker done -> B bridge -> B client receive -> pending becomes false.
```

If C deleted the task before this edit was processed, findTask throws. B receives an error and no edit state is committed. The draft is not automatically restored after an accepted send.

# Delete in C | Click and send

## Delete 1 - Click the selected row

File: Project-C/src/app/app.html

The button passes the stable task ID, not the task's position in the list.

```
<button type="button" (click)="deleteTask(task.id)"
        [disabled]="!state.connected || state.pending">
  Delete
</button>
```

Next: Project-C/src/app/app.ts -> deleteTask(1).

## Delete 2 - Call the loaded client

File: Project-C/src/app/app.ts

The component delegates the action to A's client implementation running in C's tab.

```
deleteTask(id: number) {
  this.appState()?.deleteTodo(id);
}
```

Next: Project-A/src/app/app-state.service.ts -> deleteTodo(1).

## Delete 3 - Create and deliver the command

File: Project-A/src/app/app-state.service.ts

send attaches a request ID, marks C pending and posts to its bridge. The bridge forwards the action to the worker.

```
deleteTodo(id: number) {
  return this.send({ type: 'deleteTodo', id });
}
// Inner action: { type: 'deleteTodo', id: 1 }
```

Next: Project-A/public/state-worker.js -> port.onmessage -> applyAction().

# Delete in C | Remove and refresh

## Delete 4 - Check existence and remove

File: Project-A/public/state-worker.js

applyAction first clones the latest state. findTask confirms that ID 1 exists. filter builds an array without that ID.

```
case 'deleteTodo':
  findTask(state, action.id);
  state.todos = state.todos.filter(task => task.id !== action.id);
  break;

// Before: todos = [{ id: 1, text: 'Present demo', completed: false }]
// After:  todos = []
store.next(state);
```

Next: The RxJS subscriber broadcasts the state containing the shorter list.

## Delete 5 - Every screen removes the row

```
state-worker.js: store.subscribe()
  -> state-bridge.js: port.onmessage -> sendToParent()
  -> app-state.service.ts: receive() -> validState() -> publish()
  -> state$ emits the new list
  -> A/todo.html, B/app.html and C/app.html use AsyncPipe
  -> Angular removes task 1 from each list.
```

## Delete 6 - C finishes waiting

```
{ type: 'done', requestId: 'req-C-1' }
```

The worker sends this only to C's requesting connection. C's receive matches the ID, clears the timeout and sets pending:false. If this was the last task, each list shows its empty message. The next task ID is not decremented by deletion.

# Counter B | Click + and create the action

Starting example: countB = 2 and countC = 5. This counter scenario is independent of the earlier todo example; the current todo list is preserved.

## Counter 1 - Click Increase

File: Project-B/src/app/app.html

The + button calls increment. It is disabled until the client is connected and has no outstanding action.

```
<button type="button" (click)="increment()"
        [disabled]="!state.connected || state.pending"
        aria-label="Increase count">+</button>
```

Next: Project-B/src/app/app.ts -> increment().

## Counter 2 - Select B and the operation

File: Project-B/src/app/app.ts

The component asks for an increment. It does not calculate and send a replacement value of 3.

```
increment() {
  this.appState()?.updateCounter('b', 'increment');
}
```

Next: Project-A/src/app/app-state.service.ts -> updateCounter("b", "increment").

## Counter 3 - Package the command

File: Project-A/src/app/app-state.service.ts

The client turns those arguments into an action object.

```
updateCounter(project: 'b' | 'c',
              action: 'increment' | 'decrement' | 'reset') {
  return this.send({ type: 'counter', project, action });
}
```

Next: send() receives { type: "counter", project: "b", action: "increment" }.

# Counter B | Deliver the exact message

## Counter 4 - Send with a request ID

File: Project-A/src/app/app-state.service.ts

send checks connected/pending, creates the UUID, sets pending:true, starts its timer and sends to B's hidden A-origin iframe.

```
{
  channel: 'project-a-state',
  type: 'action',
  requestId: 'req-B-1',
  action: { type: 'counter', project: 'b', action: 'increment' }
}
```

Next: state-bridge.js -> window message listener receives this envelope.

## Counter 5 - Relay it to the common worker

File: Project-A/public/state-bridge.js

After validating source/origin/channel, the bridge forwards the same message over its worker connection.

```
if (message.type === 'action' && worker)
  worker.port.postMessage(message);
```

Next: state-worker.js -> port.onmessage receives message.action.

## Counter 6 - Choose the counter branch

File: Project-A/public/state-worker.js

The handler calls applyAction. That function copies store.value and selects the counter case.

```
const state = structuredClone(store.value);
// Inside switch (action.type):
case 'counter':
  updateCounter(state, action);
  break;
```

Next: updateCounter(state, action) in the worker changes the copied number.

The client and worker both have a function named updateCounter. The client function sends a request. The worker function performs the actual change.

# Counter B | Modify and broadcast

## Counter 7 - Calculate from the latest state

File: Project-A/public/state-worker.js

This is where 2 becomes 3. The helper validates the project/action first and checks that the result is a safe integer.

```
function updateCounter(state, action) {
  if (!['b', 'c'].includes(action.project) ||
      !['increment', 'decrement', 'reset'].includes(action.action)) {
    throw new Error('Invalid counter action.');
  }
  const key = action.project === 'b' ? 'countB' : 'countC';
  if (action.action === 'increment') state[key]++;
  if (action.action === 'decrement') state[key]--;
  if (action.action === 'reset') state[key] = 0;
  if (!Number.isSafeInteger(state[key])) throw new Error('Counter limit reached.');
}
```

Next: Return to applyAction(); no page computes the shared replacement number.

## Counter 8 - Publish the new value

File: Project-A/public/state-worker.js

After the helper returns, applyAction calls store.next. The subscription sends the full state to every connected port.

```
store.next(state);

// Snapshot payload; existing todos are included unchanged:
{ countB: 3, countC: 5, todos: /* current task list */ [] }

// Each port receives:
{ type: 'state', state }
```

Next: Each bridge receives the state and calls sendToParent().

The [] above is an illustrative empty list. If tasks exist, the message contains those tasks. Updating B never intentionally resets C or clears todos.

# Counter B | Display and acknowledge

## Counter 9 - Forward and publish the reply

File: Project-A/src/app/app-state.service.ts

Each bridge calls sendToParent. Each client's receive checks the message, validates the snapshot and calls publish. Its state$ emits countB:3 and countC:5.

```
this.publish({
  ...message.state,
  connected: true,
  pending: this.view.pending,
  error: ''
});
```

Next: A and B's AsyncPipe bindings receive the new view.

## Counter 10 - Render the values

File: Project-A/src/app/app.html

A displays both counts. B displays state.countB in its own app.html. C receives the snapshot too, but its counter displays countC, which remains 5.

```
<!-- A's display expression -->
{{ state.connected ? state.countB : '-' }}

<!-- B's app.html also reads: -->
{{ state.connected ? state.countB : '-' }}
```

Next: A shows B = 3 and C = 5; B shows 3; C shows 5. The disconnected dash is normalized in this excerpt.

## Counter 11 - Confirm B's action

File: Project-A/src/app/app-state.service.ts

The worker queues done after broadcasting. Through B's bridge, it reaches receive. Only the matching request ID ends B's pending action.

```
// Matching done branch:
clearTimeout(this.replyTimeout);
this.pendingRequest = undefined;
this.publish({ ...this.view, pending: false });
```

Next: B's counter buttons become available again.

# Counters | Decrease, reset and simultaneous tabs

## Decrease B or change C

File: Project-C/src/app/app.ts

B uses project b; C uses project c. All commands follow the entire counter path just described: component -> client -> bridge -> worker -> every screen.

```
// C's methods:
increment() { this.appState()?.updateCounter('c', 'increment'); }
decrement() { this.appState()?.updateCounter('c', 'decrement'); }
reset()     { this.appState()?.updateCounter('c', 'reset'); }

// B's decrease method passes:
this.appState()?.updateCounter('b', 'decrement');
```

Next: The worker chooses countB or countC, modifies it and calls store.next().

## Concrete before-and-after values

```
Start:       B = 2, C = 5
B increase:  B = 3, C = 5
B decrease:  B = 2, C = 5
C increase:  B = 2, C = 6
C decrease:  B = 2, C = 5
C reset:     B = 2, C = 0
C decrease:  B = 2, C = -1   (negative counts are allowed)
```

## Two B tabs press + nearly together

If countB starts at 2, each tab sends an increment operation. The worker processes one callback at a time: first 2 -> 3, then 3 -> 4. Each action reads the latest store.value. Both tabs eventually show 4; they do not both submit a stale replacement value of 3.

A state update does not finish another request. If B is waiting on its save and receives C's counter update, B keeps pending:true until its own matching done/error arrives.

# Complete a task | Checkbox flow

## Complete 1 - Send the selected ID

File: Project-A/src/app/todo.ts

The checkbox in todo.html calls toggleTask(task.id). The client sends an operation rather than a locally calculated completion value.

```
toggleTask(id: number) {
  appState.toggleTodo(id);
}
// Client method:
toggleTodo(id: number) {
  return this.send({ type: 'toggleTodo', id });
}
```

Next: send -> bridge listener -> worker port.onmessage -> applyAction.

## Complete 2 - Flip the latest value

File: Project-A/public/state-worker.js

The worker finds the task in its copied state and reverses completed.

```
case 'toggleTodo': {
  const task = findTask(state, action.id);
  task.completed = !task.completed;
  break;
}
// false -> true on the first accepted toggle
// true -> false on the next accepted toggle
```

Next: store.next -> store.subscribe -> bridge sendToParent -> client receive/publish -> state$ -> AsyncPipe.

## Complete 3 - Every screen reflects completion

A's checkbox uses [checked]="task.completed". All task displays use [class.completed]="task.completed" to apply completed styling. The text and ID stay unchanged. The worker sends done only to the requesting A client.

# When something fails or a tab closes

## A task was deleted before B saved

File: Project-A/public/state-worker.js

findTask cannot find the requested ID and throws. The handler returns error instead of done. No edit state is committed.

```
catch (error) {
  port.postMessage({
    type: 'error', requestId: message.requestId, message: error.message
  });
}
```

Next: Bridge -> client receive -> showError -> publish -> template displays the error.

## There is no timely reply

File: Project-A/src/app/app-state.service.ts

waitForReply starts a 10-second timer. On expiry showError clears the pending request and marks the client disconnected. It does not cancel or undo an action the worker may have processed.

```
this.replyTimeout = setTimeout(() => {
  this.showError(
    'No reply from the shared store. Start Project A on port 4200 and reload.',
    true
  );
}, 10000);
```

Next: Inspect the current state after reconnecting before repeating an uncertain action.

## Reloading and closing tabs

Reload A while B/C remain connected: the new A client receives the existing worker state. Close A's tab: B/C can keep their bridges connected. Keep A's asset server running for new loads. Closing the last connected bridge normally makes the worker close, losing its in-memory data. Different browser profiles/computers do not share this store.

There is no localStorage or backend business-data service in this flow. Federation dependency singletons do not provide cross-tab memory. The UI roles are not authentication permissions.

# Demo checklist and the complete call chain

## A presentation sequence you can follow

Open A, B and C in the same browser profile using localhost ports 4200, 4201 and 4202.

Add "Prepare demo" in A and show it in B/C.

Edit to "Present demo" in B and show the changed text in A/C.

Complete the task in A and show the styling in B/C.

Increase B and decrease C; point to A's summary.

Delete the task in C and show all lists updating.

Add another task, then reload A while B/C remain connected. Explain that the worker supplies the current data.

## The full chain for any successful change

```
Component function
 -> client action method -> send()
 -> bridge window message listener -> worker port.onmessage
 -> applyAction() -> helper / requested mutation -> store.next()
 -> store.subscribe() -> every bridge's port.onmessage
 -> sendToParent() -> each client's receive() -> validState()
 -> publish() -> state$ -> AsyncPipe -> Angular updates the screen

Separate confirmation:
worker done -> requesting bridge -> requesting client receive()
 -> matching request ID -> pending:false
```

## What to say aloud

"Our components send actions through a federated client. A hidden bridge connects each tab to one worker. The worker updates the RxJS store and sends the new state to all connected tabs. Their observables emit, and AsyncPipe lets Angular refresh the screens."

Source basis: current component/template files in Projects A/B/C, A's federation configuration, app-state.service.ts, state-bridge.js and state-worker.js. Source excerpts are intentionally focused; this document changes no application code.